

Trabalho Prático 1 • Prof. Carlos Eduardo C. F. Batista

Nome:	Matrícula:	Valor: 3,0 pts	Extra: +1,0 pt
-------	------------	----------------	----------------

Neste trabalho você irá implementar um pequeno sistema orientado a objetos sobre um **tema de sua escolha**. O objetivo é consolidar encapsulamento, composição, agregação e boas práticas de C++17 vistas em aula.

Prazo: 28/05/2026 até as 12h15

Entrega: e-mail para bidu@ci.ufpb.br com assunto exato: [POO-261-T001] Nome - Matrícula

Anexo: ZIP do repositório Git ou URL pública do GitHub

Ferramentas: LLMs são permitidos — siga o Guia de Uso Inteligente de IA já distribuído

⚠ Plágio: Trabalhos idênticos ou com trechos copiados de colegas resultam em nota zero para todos os envolvidos, independentemente de quem enviou primeiro. Uso de LLM é permitido; copiar o trabalho de outro aluno, não.

Questão 1 — Repositório e Estrutura do Projeto (1,0 ponto)

Configure o repositório e a estrutura de build do projeto.

(A) Crie um repositório Git (local ou no GitHub) com a seguinte estrutura mínima:

```
meu-projeto/  
├── CMakeLists.txt      (ou Makefile)  
├── README.md  
└── src/  
    ├── main.cpp  
    └── (demais arquivos .cpp / .hpp)
```

(B) O projeto deve compilar sem erros com: `cmake -B build && cmake --build build` (ou `make`).

(C) O `README.md` deve conter: (1) nome e matrícula; (2) descrição do domínio escolhido (3–5 linhas); (3) diagrama UML de classes em **Mermaid**, SVG ou imagem, com todas as classes, atributos, métodos e relações.

Exemplo de diagrama Mermaid no README:

```
```mermaid  
classDiagram
 class Pedido {
 -int id_
 -float valor_total
 +calcular_total() float
 }
 class Item {
 -string nome_
 -float preco_
 }
 Pedido "1" *-- "1..*" Item : contém
```
```

Questão 2 — Classes e Encapsulamento (1,0 ponto)

Implemente pelo menos 4 classes que modelem o domínio escolhido, seguindo as convenções da disciplina.

- (A) Cada classe deve ter: membros privados com sufixo `_`; construtores com lista de inicialização; getters `const` para os atributos que precisam ser lidos externamente.
- (B) Pelo menos **uma classe** deve ter um destrutor explícito que produza algum efeito observável (ex.: imprime mensagem, libera recurso gerenciado).
- (C) Pelo menos **um método por classe** deve implementar lógica real — não pode ter corpo vazio. Exemplos: calcular valor, formatar saída, validar dado, atualizar estado.
- (D) No `main()`, instancie pelo menos um objeto de cada classe e demonstre pelo menos um método com lógica real sendo executado, com saída visível no terminal.

Convenções obrigatórias: snake_case em tudo • membros privados com sufixo `_` • identificadores em inglês • comentários em português

Questão 3 — Composição e Agregação (1,0 ponto)

Implemente as relações entre classes do seu domínio. O sistema deve conter **pelo menos uma composição e pelo menos uma agregação**.

(A) Na **composição**: a classe dona deve criar o objeto dependente (em seu construtor ou em método próprio) e destruí-lo junto consigo. O dependente não existe sem o dono. Reflita isso no destrutor ou na estrutura dos membros.

(B) Na **agregação**: a classe principal referencia objetos que existem independentemente dela. O destrutor da classe principal **não deve deletar** os objetos agregados.

(C) No **README.md**, identifique no diagrama UML qual relação é composição (◆) e qual é agregação (◇), com uma frase justificando cada escolha em termos de ciclo de vida.

(D) No **main()**, demonstre as duas relações: crie os objetos, use-os e mostre (via saída no terminal) o que acontece quando o objeto dono é destruído.

Exemplo de saída esperada na demonstração de composição:

```
Pedido(001) criado
  Item("Caneta") criado
  Item("Caderno") criado
Total do pedido: R$ 18.50
~Item("Caderno") destruido
~Item("Caneta") destruido
~Pedido(001) destruido
```

Questão 4 — Smart Pointers (extra) (+1,0 ponto)

Esta questão tem pontuação extra. Substituir raw pointers por smart pointers nos pontos corretos do seu projeto.

(A) Identifique todos os raw pointers do seu projeto. Para cada um, aplique **unique_ptr** ou **shared_ptr** conforme a semântica de posse, eliminando destrutores manuais onde for possível.

(B) No **README.md**, adicione uma seção "Smart Pointers" explicando: qual ponteiro você usou em cada relação e por quê (uma linha por decisão).

(C) O projeto deve continuar compilando e produzindo a mesma saída após a substituição.

Dica: composição → **unique_ptr** • recurso genuinamente compartilhado → **shared_ptr** • observador sem posse → referência &

Roteiro de Autoavaliação

Antes de enviar, marque cada item abaixo. Eles correspondem diretamente aos critérios de correção. **Se todos estiverem marcados, você atingiu a nota máxima obrigatória.**

| | Critério | Pts |
|--------------------------|---|-------|
| ☐ | Q1 — Repositório e estrutura | 1,0 |
| ☐ | → Repositório Git criado com estrutura de diretórios organizada | 0,2 |
| ☐ | → CMakeLists.txt ou Makefile presente e funcional (compila sem erros) | 0,3 |
| ☐ | → README.md com nome, matrícula, descrição do domínio e diagrama UML | 0,3 |
| ☐ | → Diagrama UML contém todas as classes, atributos, métodos e relações | 0,2 |
| ☐ | Q2 — Classes e encapsulamento | 1,0 |
| ☐ | → Pelo menos 4 classes implementadas | 0,2 |
| ☐ | → Todos os membros privados com sufixo _ e getters const | 0,2 |
| ☐ | → Construtores usam lista de inicialização | 0,2 |
| ☐ | → Pelo menos um destrutor explícito e observável | 0,2 |
| ☐ | → Pelo menos um método com lógica real por classe; saída no terminal | 0,2 |
| ☐ | Q3 — Composição e agregação | 1,0 |
| ☐ | → Pelo menos uma composição implementada corretamente | 0,25 |
| ☐ | → Pelo menos uma agregação implementada corretamente | 0,25 |
| ☐ | → Destrutores refletem o ciclo de vida correto de cada relação | 0,25 |
| ☐ | → README justifica as relações; UML usa ◆ e ◇ corretamente | 0,25 |
| ☐ | Q4 — Smart pointers (extra) | +1,0 |
| ☐ | → Raw pointers substituídos por unique_ptr ou shared_ptr corretamente | +0,5 |
| ☐ | → Destrutores manuais eliminados onde possível | +0,25 |
| ☐ | → README explica cada decisão de smart pointer | +0,25 |

Roteiro de Testes Manuais

Execute os passos abaixo para verificar se sua implementação está correta. **Não é necessário Catch2** — os testes são feitos diretamente no `main()` e validados pela saída no terminal.

Teste 1 — Compilação limpa

```
cmake -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build
# Esperado: zero erros, zero warnings -Wall -Wextra
```

Teste 2 — Construtores e destrutores

Adicione mensagens de `std::cout` em cada construtor e destrutor de suas classes. Execute o programa e verifique:

- Cada objeto criado imprime sua mensagem de construção na ordem esperada.
- Ao final do `main()`, os objetos são destruídos na ordem inversa.
- Objetos em composição são destruídos **junto com** o dono; objetos em agregação sobrevivem à destruição da classe principal.

Teste 3 — Encapsulamento

Tente, temporariamente, acessar um membro privado diretamente fora da classe:

```
objeto.nome_ = "teste";
// Esperado: ERRO de compilação — membro privado não acessível
// Desfaça após confirmar o erro
```

Teste 4 — Lógica dos métodos

Para cada método com lógica real, verifique com pelo menos dois casos:

- Um caso normal, com saída esperada calculada manualmente antes de executar.
- Um caso limite (valor zero, string vazia, lista vazia etc.) — o programa não deve travar.

Teste 5 — Composição (ciclo de vida)

No `main()`, crie o objeto dono dentro de um bloco `{ }` e verifique que os objetos dependentes são destruídos ao sair do bloco:

```
{
    MinhaClasseDona dono(...);
    // usa dono
} // <-- destrutor deve aparecer aqui, com os dependentes
std::cout << "Fora do bloco\n";
// Esperado: mensagens de destruição ANTES de "Fora do bloco"
```

Teste 6 — Agregação (independência)

Crie os objetos agregados **fora** da classe principal e confirme que continuam acessíveis após a destruição da classe principal:

```
ObjetoAgregado agregado(...);
{
    ClassePrincipal principal(..., &agregado);
} // principal destruída aqui
agregado.algum_metodo();
// Esperado: executa normalmente — agregado ainda existe
```

Teste 7 — Smart pointers (apenas Q4)

Compile com AddressSanitizer e verifique que não há vazamentos:

```
cmake -B build -DCMAKE_BUILD_TYPE=Debug
# Adicione ao CMakeLists.txt em modo Debug:
# add_compile_options(-fsanitize=address,undefined)
# add_link_options(-fsanitize=address,undefined)
cmake --build build && ./build/meu_exe
# Esperado: nenhuma linha "ERROR: AddressSanitizer"
```